

Mauricio de Souza Coutinho

Exercício 1:

Liste nome , preço e preço com desconto de 12%.

- `CREATE DATABASE loja_exercicio;`
- `USE loja_exercicio;`
- `-- Criação da tabela de produtos`
- `CREATE TABLE produtos (
 id INT AUTO_INCREMENT PRIMARY KEY,
 nome VARCHAR(100) NOT NULL,
 preco DECIMAL(10, 2) NOT NULL,
 categoria VARCHAR(50) NOT NULL,
 situacao_estoque VARCHAR(50) NOT NULL
);`
- `SELECT
 nome AS Nome_Produto,
 preco AS Preco_Original,
 (preco * 0.88) AS Preco_Com_Desconto_12
FROM produtos;`

Mostre as combinações distintas de categoria e situação de estoque.

	Nome_Produto	Preco_Original	Preco_Com_Desconto_12
▶	Notebook Gamer	4500.00	3960.0000
	Smartphone X	2500.00	2200.0000
	Cadeira Ergonômica	1200.00	1056.0000
	Mesa de Escritório	850.00	748.0000
	Fone de Ouvido Bluetooth	300.00	264.0000
	Mouse Sem Fio	120.00	105.6000
	Estante para Livros	450.00	396.0000
	Monitor 27 Polegadas	1400.00	1232.0000

- `SELECT DISTINCT
 categoria,
 situacao_estoque
FROM produtos;`

	categoria	situacao_estoque
▶	Eletrônicos	Em Estoque
	Móveis	Baixo Estoque
	Móveis	Esgotado
	Eletrônicos	Baixo Estoque

Explique o que o DISTINCT considera duplicado:

O comando DISTINCT considera uma linha como duplicada se todos os valores selecionados nas colunas especificadas forem idênticos aos de outra linha. No caso acima, ele vai agrupar e retornar apenas pares únicos de categoria e situação de estoque.

Exercício 2:

Encontre clientes ativos cujo nome começa com A ou B

Usamos operadores lógicos, **LIKE** para os padrões de início de nome e parênteses para organizar a prioridade:

```
CREATE TABLE clientes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    ativo BOOLEAN NOT NULL, -- 1 para ativo, 0 para inativo  
    telefone VARCHAR(20) NULL,  
    email VARCHAR(100) NULL  
);
```

```
SELECT *  
FROM clientes  
WHERE ativo = 1  
AND (nome LIKE 'A%' OR nome LIKE 'B%');
```

	id	nome	ativo	telefone	email
▶	1	Ana Souza	1	NULL	ana.souza@email.com
	2	Bruno Mendes	1	11988887777	bruno@email.com
	4	Amanda Dias	1	11977776666	NULL
	5	Beatriz Lima	1	NULL	beatriz@email.com
*	NULL	NULL	NULL	NULL	NULL

Liste produtos de categorias selecionadas com preço entre 10 e 80

Usamos o operador BETWEEN (ou >= e <=) junto com IN para as categorias:

```
SELECT *  
FROM produtos  
WHERE categoria IN ('Acessórios', 'Eletrônicos')  
AND preco BETWEEN 10.00 AND 80.00;
```

	id	nome	preco	categoria	situacao_estoque
▶	9	Mouse Pad	25.50	Acessórios	Em Estoque
	11	Cabo HDMI	45.00	Acessórios	Em Estoque
	12	Suporte para Monitor	79.90	Acessórios	Baixo Estoque
*	NULL	NULL	NULL	NULL	NULL

Localize clientes sem telefone e com e-mail informado

Aqui tratamos diretamente o conceito de NULL:

```
SELECT *  
FROM clientes  
WHERE telefone IS NULL  
AND email IS NOT NULL
```

	id	nome	ativo	telefone	email
▶	1	Ana Souza	1	NULL	ana.souza@email.com
	3	Carlos Silva	0	NULL	carlos@email.com
	5	Beatriz Lima	1	NULL	beatriz@email.com
*	NULL	NULL	NULL	NULL	NULL

Crie uma consulta que combine AND e OR; use parênteses

Exemplo prático: Buscar produtos que sejam da categoria 'Acessórios' e que custem menos de R\$ 50,00 ou que tenham a situação 'Baixo Estoque' independentemente da categoria.

<code>SELECT *</code>	
<code>FROM produtos</code>	
<code>WHERE (categoria = 'Acessórios' AND preco < 50.00)</code>	
<code>OR situacao estoque = 'Baixo Estoque';</code>	

id	nome	preco	categoria	situacao_estoque
3	Cadeira Ergonômica	1200.00	Móveis	Baixo Estoque
6	Mouse Sem Fio	120.00	Eletrônicos	Baixo Estoque
9	Mouse Pad	25.50	Acessórios	Em Estoque
11	Cabo HDMI	45.00	Acessórios	Em Estoque
12	Suporte para Monitor	79.90	Acessórios	Baixo Estoque
NULL	NULL	NULL	NULL	NULL

Nota pedagógica: O uso de parênteses é obrigatório para alterar a precedência padrão do SQL (onde o AND costuma ser executado antes do OR), garantindo que a lógica da regra de negócio fique correta.

Provoque o erro conceitual = NULL e explique o resultado

Se você tentar buscar clientes sem telefone usando o sinal de igual = da seguinte forma:

<code>SELECT *</code>	
<code>FROM clientes</code>	
<code>WHERE telefone = NULL;</code>	

id	nome	ativo	telefone	email
NULL	NULL	NULL	NULL	NULL

Explicação do resultado: Em banco de dados relacionais, NULL não representa um valor ou um espaço vazio comum, mas sim a **ausência de dado** (um valor desconhecido). Como o SQL opera com lógica ternária (Verdadeiro, Falso e Desconhecido), qualquer comparação direta usando operadores de igualdade (= NULL) ou diferença (≠ NULL) resulta em "Desconhecido", fazendo com que a linha seja ignorada. Por isso, a forma correta de verificar a ausência de dados é estritamente com os operadores **IS NULL** ou **IS NOT NULL**.

Exercício 3:

Mostre os dez produtos de maior valor em estoque (com desempate por nome e justificativa)

Para calcular o "valor em estoque" de um produto, podemos multiplicar o preço pela quantidade (ou considerar o preço ordenado de forma decrescente caso o estoque seja unitário/tabela simples). Vamos assumir a ordenação pelo preço de forma decrescente com o desempate por nome.

<code>SELECT</code>	
<code>nome,</code>	
<code>preco,</code>	
<code>situacao_estoque</code>	
<code>FROM produtos</code>	
<code>ORDER BY preco DESC, nome ASC</code>	
<code>LIMIT 10;</code>	

nome	preco	situacao_estoque
Notebook Gamer	4500.00	Em Estoque
Smartphone X	2500.00	Em Estoque
Monitor 27 Polegadas	1400.00	Em Estoque
Cadeira Ergonômica	1200.00	Baixo Estoque
Mesa de Escritório	850.00	Esgotado
Estante para Livros	450.00	Esgotado
Fone de Ouvido Bluetooth	300.00	Em Estoque
Teclado Mecânico	150.00	Em Estoque
Caixa de Som	120.00	Em Estoque
Mouse Sem Fio	120.00	Baixo Estoque

Justificativa da ordem escolhida: Ordenamos primeiro por preço DESC (do mais caro para o mais barato) para atender à regra principal de maior valor. Em seguida, adicionamos nome ASC (ordem alfabética crescente) como critério de **desempate**. Isso garante um comportamento determinístico: se dois produtos tiverem exatamente o mesmo preço, eles sempre aparecerão na mesma ordem previsível (de A a Z) em todas as execuções, evitando que o SGBD exiba linhas de forma aleatória nas páginas seguintes.

Crie uma segunda página com LIMIT e OFFSET

Assumindo que cada página exibe 5 registros (LIMIT 5), para acessar a segunda página precisamos pular os registros da primeira página usando o OFFSET 5:

<pre>• SELECT id, nome, preco FROM produtos ORDER BY id LIMIT 5 OFFSET 5;</pre>		id	nome	preço
	▶	6	Mouse Sem Fio	120.00
		7	Estante para Livros	450.00
		8	Monitor 27 Polegadas	1400.00
		9	Mouse Pad	25.50
		10	Teclado Mecânico	150.00
	✱	NULL	NULL	NULL

Explicação: O LIMIT 5 restringe o retorno a 5 linhas por página, e o OFFSET 5 pula as primeiras 5 linhas (que formavam a página 1), entregando os registros da página 2.

Formate uma data de pedido e calcule os dias decorridos

No MySQL, podemos usar a função DATE_FORMAT para formatar a data e a função DATEDIFF para calcular a diferença de dias entre a data atual (NOW() ou CURRENT_DATE) e a data do pedido:

<pre>• SELECT id, DATE_FORMAT(data_pedido, '%d/%c/%Y') AS data_formatada, DATEDIFF(CURRENT_DATE(), data_pedido) AS dias_decorridos FROM pedidos;</pre>		id	data_formatada	dias_decorridos
	▶	1	15/1/2026	264
		2	10/2/2026	238
		3	01/3/2026	219
		4	20/3/2026	200
		5	05/4/2026	184

Explicação: DATE_FORMAT converte a data padrão do banco (AAAA-MM-DD) para o formato brasileiro (DD/MM/AAAA), e DATEDIFF subtrai a data do pedido da data atual, retornando o número exato de dias passados.

Compare CHAR_LENGTH e LENGTH usando texto com acento

Podemos usar uma consulta de teste utilizando o comando SELECT sem tabela física para entender a diferença prática entre essas duas funções de string do MySQL:

- **SELECT**

```
'Ação' AS palavra,
CHAR_LENGTH('Ação') AS quantidade_caracteres,
LENGTH('Ação') AS quantidade_bytes;
```

	palavra	quantidade_caracteres	quantidade_bytes
	Ação	4	6

Explicação do resultado:

CHAR_LENGTH(): Conta o número de caracteres visíveis da string. Para 'Ação', o resultado retornado será 4.

LENGTH(): Conta o número de bytes que a string ocupa no armazenamento (considerando o charset UTF-8). Como a letra ç com acento (ou a própria letra ã) costuma ocupar 2 bytes em UTF-8 no MySQL, o resultado de LENGTH('Ação') será 5 (1 byte para o 'A', 1 para o 'c', 2 para o 'ç' e 1 para o 'o').

Desafio Integrador:

Duas consultas de projeção , aliases , expressões ou DISTINCT:

- **SELECT**

```
nome AS Nome_Produto,
preco AS Preco_Original,
(preco * 0.88) AS Preco_Com_Desconto_12
FROM produtos;
```

- **SELECT DISTINCT**

```
categoria,
situacao_estoque
FROM produtos;
```

Resultado Esperado 1 : Uma tabela com três colunas (Nome_Produto, Preco_Original, Preco_Com_Desconto_12), exibindo os valores calculados linha a linha.

Interpretação 1: A projeção seleciona colunas específicas, o alias (AS) melhora a legibilidade do cabeçalho e a expressão matemática aplica o fator multiplicativo direto (0.88) para o desconto de forma individual por registro.

Resultado Esperado 2 : Um conjunto resumido de linhas contendo apenas os pares únicos de categoria e situação de estoque cadastrados.

Interpretação 2 : O DISTINCT avalia a combinação integral das colunas informadas e remove todas as linhas duplicadas, filtrando a visualização para registros exclusivos.

Duas consultas com filtros , padrões , intervalos e tratamento de NULL:

- **SELECT ***

```
FROM clientes
WHERE ativo = 1
AND (nome LIKE 'A%' OR nome LIKE 'B%');
```

- **SELECT ***

```
FROM clientes
WHERE telefone IS NULL
AND email IS NOT NULL
```

Resultado Esperado 1 : Apenas os registros de clientes que possuem status ativo (1) e cujos nomes comecem estritamente com 'A' ou 'B'.

Interpretação 1 : O uso de parênteses garante a precedência lógica correta, assegurando que o filtro de status (ativo = 1) seja obrigatório para qualquer uma das condições de correspondência de padrão de texto (LIKE).

Resultado Esperado 2: Linhas correspondentes aos clientes cuja coluna de telefone está vazia/nula no banco de dados.

Interpretação 2: Demonstra o tratamento correto de ausência de dados empregando o operador IS NULL, evitando o erro conceitual de utilizar o sinal de igualdade (= NULL), que resultaria em um retorno vazio por conta da lógica ternária do SQL.

Duas consultas com ordenação , limites e funções:

- ```
SELECT nome, preco
FROM produtos
ORDER BY preco DESC, nome ASC
LIMIT 5;
```

Resultado Esperado: Exatamente os 5 produtos mais caros da tabela, ordenados do maior para o menor preço, com critério de desempate alfabético por nome.

Interpretação: O ORDER BY ... DESC estabelece a fronteira superior de preços e o modificador LIMIT 5 restringe rigorosamente o escopo da saída aos cinco primeiros registros do conjunto ordenado.